

SPOTIFY PROJECT

Final Technical Report

WEB PROGRAMMING	S2026
STACK	React + Django REST
DELIVERY	Docker + PWA + Realtime

Prepared from the repository implementation and Git history. Contributor responsibilities below are inferred from committed work and should be confirmed by the team before academic submission.

1. Team and responsibilities

Contributor	Repository evidence	Primary responsibility (inferred)
JimmyNotron	34 commits	Application foundation, authentication, profiles, subscriptions, shared frontend structure and tests.
Alimirzaei84	21 commits	Music domain, player, albums, playlists, artist works, playback integration and sample data.
Mahyar	20 commits	Operational backend, payments, notifications, support, PWA, Docker, realtime group listening and delivery tooling.

Working conventions

- Feature-oriented React components and service modules; Django apps split by business domain.
- REST payloads use snake_case at the API boundary and are mapped to camelCase UI models.
- Protected operations rely on JWT authentication and server-side role or subscription permissions.
- Schema migrations accompany model changes; focused tests accompany API and frontend service contracts.
- Git history uses small feature/fix commits and keeps generated secrets outside version control.

Maintainability

The system separates view rendering, HTTP services, serializers, permissions, domain services and persistence. Reusable guards and entitlement selectors avoid duplicating access rules. Environment variables configure databases, Redis, CORS and Zarinpal endpoints, while Docker provides a reproducible deployment topology.

2. Architecture

Layer	Technology	Responsibilities
Client	React 19, TypeScript, Vite	Responsive pages, player state, forms, PWA shell and WebSocket client.
API	Django 5.2, DRF	Authentication, validation, CRUD, roles, subscriptions, reports and payment orchestration.
Realtime	Django Channels, Redis	Temporary listening rooms, membership and synchronized playback events.
Data	PostgreSQL	Users, preferences, catalog, streams, tickets, notifications, transactions and rooms.
Edge	Nginx	Static frontend delivery and reverse proxying for HTTP, media and WebSockets.

Request flow

Browser requests enter Nginx. Static navigation is served by the React application; /api requests are routed to Django/Daphne; /ws connections are upgraded to Channels. Django authenticates JWTs, applies role and plan permissions, validates serializers and persists through PostgreSQL. Redis carries realtime channel messages.

Deployment

compose.yaml starts frontend Nginx, Django/Daphne, PostgreSQL and Redis. The backend entrypoint waits for its dependencies, applies migrations and collects static assets. Persistent volumes protect database and uploaded-media data.

PWA

The frontend includes a web manifest, application icons and a registered service worker. The application shell and safe static assets are cached for installability and offline startup without caching authenticated API responses.

3. Domain model relationships

Aggregate	Key relationships
Identity	User 1:1 UserPreference; verified artist 1:1 ArtistProfile; user 1:N ArtistApplication reviews.
Subscriptions	SubscriptionPlan 1:N UserSubscription and SubscriptionOrder; orders produce gateway transaction state.
Music	Artist(User) 1:N Album and Song; Album 1:N Song; Playlist M:N Song through ordered PlaylistSong.
Listening	User and Song create Stream/RecentlyPlayed records; ListeningRoom has memberships and a current Song.
Support	User 1:N SupportTicket; ticket 1:N TicketMessage; support/admin may be assigned to tickets.
Operations	User 1:N Notification; artists receive monthly payout/report rows derived from stream activity.

Integrity and access

- Object ownership is checked for song, album and playlist mutations.
- Only active, verified artist accounts may publish or modify music.
- Audio uploads accept MP3, WAV or FLAC MIME/extension pairs and are capped at 50 MB.
- Support users and administrators can manage tickets; only administrators can change subscription prices.
- Subscription entitlements enforce playlist, streaming, early-access, statistics and download features.

4. Implemented feature coverage

Requirement group	Implementation
Authentication	Listener/artist registration, login, JWT refresh/logout, privacy acknowledgement and password reset.
Discovery and profiles	Home, search, recent playlists, albums/singles, user/artist profiles, follows and verification badge.
Playback	Play/pause, seek, next/previous, volume, queue reorder, shuffle, repeat, lyrics and stream tracking.
Artist tools	Verified-artist album/song CRUD, covers, metadata, lyrics, audio upload and statistics.
Plans and payments	Free/Silver/Gold entitlements, durations, dynamic admin prices and Zarinpal sandbox request/verification.
Operations	Backend notifications, support chat, artist review, payment audit and monthly artist reporting.
Optional	Realtime temporary group listening with invite code, synchronized state and cleanup after the last member leaves.

Quality controls

The repository contains more than the required ten frontend and fifteen backend test files. TypeScript production build succeeds. Focused tests for backend-connected notification, ticket and admin service contracts pass. Django tests are designed for PostgreSQL and the realtime suite additionally requires Channels and Redis.

Known validation note

At report generation time, the local dependency mirror could not download Channels/Gunicorn and the local PostgreSQL/Redis services were unavailable. The Docker definitions declare these dependencies, but a clean connected environment must run the complete compose smoke test and all backend tests before submission.

5. Use of AI-assisted development

Role and examples

AI assistance was used as an implementation and review aid: mapping the assignment checklist to repository features, generating repetitive serializer/service/test scaffolding, reviewing authorization boundaries, identifying frontend local-storage fallbacks, correcting stream aggregation, preparing Docker/PWA configuration and drafting this report.

Strengths

- Fast cross-layer consistency checks across React types, API payloads, Django serializers and routes.
- Useful generation of focused tests and mechanical integration code.
- Effective identification of missing validation, permissions and incomplete frontend/backend wiring.

Weaknesses and safeguards

- Generated code can appear complete while relying on mocks; repository searches and end-to-end contract tests are required.
- AI cannot prove runtime correctness without available databases, queues, dependencies and external sandboxes.
- Team identities and responsibilities cannot be reliably inferred from code alone; Git evidence must be reviewed by people.
- Security-sensitive suggestions require manual review, negative permission tests and secret-management checks.

Submission checklist

- Confirm contributor names, student IDs and responsibility descriptions.
- Run `docker compose up --build` in a connected environment and verify health, payment callback and WebSocket flow.
- Run the complete frontend and backend suites and retain the passing output.
- Review this report against institutional formatting and academic-integrity rules before submission.